



Rust for Civic Tech

Capturing, structuring, and serving Kenya's parliamentary data, in Rust.

Collins Muriuki · Mwananchi Tech

Who Am I

- Software engineer, 6 years of experience
- Backend / Distributed Systems / Developer Tooling focus
- Rustacean since 2020
- Currently building **Mwananchi Tech**, the umbrella org behind everything in this talk
- Also an Interview Engineer at **Karat**

Outside of Work

- **Video games**: currently deep into Death Stranding 2: On the Beach
- **Travel**: new places, new people, and as much time outdoors as I can get
- **BJJ**: blue belt, occasional competitor

The Journey

1. **Bunge Bits**: transcribing parliament from YouTube livestreams
2. **odnelazm**: a unified scraper for official Hansard transcripts
3. **odnelazm-cli**: ad-hoc queries and exports from the terminal
4. **odnelazm-mcp**: the same data, exposed to AI assistants
5. **odnelazm-ingest**: structuring it all into a database, plus AI enrichment
6. **Bunge Hub**: putting it in front of citizens

Act 1

Bunge Bits: Transcribing Parliament

The Problem

- Kenya's National Assembly and Senate livestream sittings on YouTube
- No official transcripts published in anything close to real time
- Goal: turn raw video into searchable summaries, automatically

Architecture: stream_pulse



A single pipeline: scrape, download, chunk, transcribe, summarize, store.

LiveStreamProcessor

```
#[derive(Debug, Clone)]
pub struct LiveStreamProcessor<D, T, S, A, P>
where
    D: DataStore + Send + Sync + 'static,
    T: Transcriber + Send + Sync + 'static,
    S: Summarizer + Send + Sync + 'static,
    A: AudioHandler + Send + Sync + 'static,
    P: ChannelScraper + Send + Sync + 'static,
{
    workdir: PathBuf,    // local fs dir for audio and transcript artefacts
    store: D,            // data storage layer
    transcriber: T,      // LLM transcription layer
    summarizer: S,       // LLM summarizer layer
    audio_handler: A,    // audio processing layer
    channel_scraper: P,  // web scraper layer
    max_streams: usize,
    chunking_config: Option<ChunkingConfig>,
}
```

Five generic traits, five swappable layers.

Running the Pipeline

```
impl<D, T, S, A, P> LiveStreamProcessor<D, T, S, A, P>
where
  D: DataStore + Send + Sync + 'static,
  T: Transcriber + Send + Sync + 'static,
  S: Summarizer + Send + Sync + 'static,
  A: AudioHandler + Send + Sync + 'static,
  P: ChannelScraper + Send + Sync + 'static,
{
  #[tracing::instrument(skip(self))]
  pub async fn run(self) → anyhow::Result<()> {
    // scrape → download audio
    // for each downloaded audio:
    //   use chunking config to decide whole-file vs chunked
    //   → transcribe → summarize → store results to db
    Ok(())
  }
}
```

A `Drop` impl cleans up the working `audio/` directory when the processor goes out of scope, win or lose.

Vendoring yt-dlp

```
// build.rs
fn main() → Result<(), Box<dyn Error>> {
    let os = env::var("CARGO_CFG_TARGET_OS")?;
    let arch = env::var("CARGO_CFG_TARGET_ARCH")?;

    let filename = match (os.as_str(), arch.as_str()) {
        ("macos", "aarch64") ⇒ "yt-dlp_macos",
        ("linux", "x86_64") ⇒ "yt-dlp_linux",
        ("windows", _) ⇒ "yt-dlp.exe",
        // ...other platforms
        _ ⇒ return Err("unsupported".into()),
    };

    // download the binary to OUT_DIR/yt-dlp, then write
    // OUT_DIR/generated.rs:
    let mut f = File::create(out_dir.join("generated.rs"))?;
    writeln!(
        f,
        "pub const YTDLP_BINARY: &[u8] = \
        include_bytes!(\"yt-dlp\");"
    )?;
}
```

```
// ytdlp.rs
#[cfg(feature = "yt-dlp-vendored")]
include!(concat!(
    env!("OUT_DIR"), "/generated.rs"
));

fn resolve_yt_dlp_binary()
→ Result<PathBuf, YtDlpError> {
    #[cfg(feature = "yt-dlp-vendored")]
    {
        // write YTDLP_BINARY to a temp
        // file, chmod +x, return path
    }

    #[cfg(not(feature = "yt-dlp-vendored"))]
    which::which("yt-dlp")
        .map_err(|_| YtDlpError::BinaryNotFound(
            "yt-dlp".into()
        ))
}
```

Rationale: ship one binary. No separate `yt-dlp` install on the host.

CLI: stream-pulse

```
#[derive(Parser)]
#[command(name = "stream-pulse")]
struct Cli {
    #[arg(long, env = "DATABASE_URL")]
    database_url: String,

    // + openai_key, max_streams, ...

    #[command(subcommand)]
    command: Command,
}

#[derive(Subcommand)]
enum Command {
    Run,
    Cron {
        #[arg(long, env = "CRON_SCHEDULE", default_value = "0 0 */4 * * *")]
        schedule: String,
    },
}
```

Cron, via Apalis

```
async fn handle_tick(_tick: Tick, config: Data<Config>) → anyhow::Result<()> {
    tracing::info!(max_streams = config.max_streams, "Running scheduled pipeline ... ");
    run_pipeline(&config).await
}

// in main():
Command::Cron { schedule } ⇒ {
    let schedule = Schedule::from_str(&schedule)?;

    let worker = WorkerBuilder::new("stream-pulse-cron")
        .backend(CronStream::new(schedule))
        .retry(RetryPolicy::retries(3))
        .layer(SentryLayer::new())
        .data(config)
        .build(handle_tick);

    worker.run().await?;
}
```

Cron { .. } and Run share the exact same run_pipeline function.

Limitations

LLM layer

- Whisper can't infer *who* is speaking
- Mangles Kenyan names
- Struggles with Swahili and mixed-language speech
- Costs add up fast, every minute of audio is billed

Scaling

- How do we handle more concurrent live streams without linearly scaling cost and compute?

Act 2

odnelazm: From Transcripts to Data

The Next Chapter

Bunge Bits transcribes audio: lossy, costly, biased toward English.

What if we used the **official transcripts** instead?

- Kenya's Parliament publishes Hansards, official transcripts of every sitting
- Hansards go back to **2006**
- Published as PDFs on `.go.ke` sites... and scraping those felt risky
- **mzalendo.com** mirrors them as clean HTML, much easier than OCR on PDFs

Birth of `odnelazm`.

Two Sources, One Cutoff

Source	Coverage	Format
<code>info.mzalendo.com</code> (archive)	2006-03-21 → 2025-07-17	HTML mirror of PDFs
<code>mzalendo.com</code> (current)	2013-03-28 → present	Structured HTML

Goal: **one unified scraper**, **one schema**, regardless of source. Coverage overlaps, routing (next slides) treats 2013-03-28 as the cutoff between them.

Example sittings: current · archive

A Unified Schema

```
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct HansardSitting {
    pub house: House,
    pub date: NaiveDate,
    pub day_of_week: String,
    pub session_type: String,
    pub time: Option<NaiveTime>,
    pub summary: Option<String>,
    pub sentiment: Option<String>,
    pub pdf_url: Option<String>,
    pub sections: Vec<HansardSection>,
}

#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct HansardSection {
    pub section_type: String,
    pub subsections: Vec<HansardSubsection>,
    pub contributions: Vec<Contribution>,
}
```

Subsections, Contributions, Members

```
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct HansardSubsection {
    pub title: String,
    pub contributions: Vec<Contribution>,
}

#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct Contribution {
    pub speaker_name: String,
    pub speaker_url: Option<String>,
    pub content: String,
    pub procedural_notes: Vec<String>,
}

#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct Member {
    pub name: String,
    pub url: String,
    pub house: House,
    pub role: Option<String>,
    pub constituency: Option<String>,
}
```

Other Scraped Data: MemberProfile

```
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct MemberProfile {
    pub name: String,
    pub slug: String,
    pub photo_url: Option<String>,
    pub biography: Option<String>,
    pub positions: Vec<String>,
    pub party: Option<String>,
    pub committees: Vec<String>,
    pub speeches_total: Option<u32>,
    pub bills: Vec<Bill>,
    pub voting_patterns: Vec<VoteRecord>,
    pub activity: Vec<ParliamentaryActivity>,
}
```

Plus pagination metadata for bills and activity (omitted here).

Routing Between Sources

```
enum ListingRoute {  
    /// Only the archive covers this range.  
    Archive,  
    /// Only the current source covers this range.  
    Current,  
    /// The range spans the cutoff: fetch from both and merge.  
    Both,  
}  
  
impl ListingRoute {  
    fn from_dates(start: Option<NaiveDate>, end: Option<NaiveDate>) → Self {  
        let cutoff = current_cutoff(); // 2013-03-28  
        match (start, end) {  
            (None, None) ⇒ ListingRoute::Current,  
            (_, Some(end)) if end < cutoff ⇒ ListingRoute::Archive,  
            (Some(start), _) if start ≥ cutoff ⇒ ListingRoute::Current,  
            _ ⇒ ListingRoute::Both,  
        }  
    }  
}
```

A plain `match` on an `Option` pair decides the whole routing strategy.

HansardScraper: the Facade

```
pub struct HansardScraper {
    archive: ArchiveScraper,
    current: CurrentScraper,
}

impl HansardScraper {
    pub async fn list_sittings(&self, opts: SittingListOptions)
        → Result<Vec<HansardListing>, ScraperError> {
        match ListingRoute::from_dates(opts.start_date, opts.end_date) {
            ListingRoute::Archive ⇒ self.fetch_archive_listings(opts).await,
            ListingRoute::Current ⇒ self.current.fetch_all_sittings(opts.house).await,
            ListingRoute::Both ⇒ {
                let (archive, current) = future::join(
                    self.fetch_archive_listings(opts),
                    self.current.fetch_all_sittings(opts.house),
                ).await;
                merge_and_sort(archive?, current?)
            }
        }
    }
}
```

`get_sitting` follows the same pattern: detect the source from the URL shape, then delegate.

CLI: Exporting Data

```
# Recent sittings (current source, page 1)
odnelazm sittings

# Archive sittings from 2010
odnelazm sittings --start-date 2010-01-01 --end-date 2010-12-31

# Cross-era range: archive + current, merged
odnelazm sittings --start-date 2012-01-01 --end-date 2014-12-31 --limit 50

# Filter by house, all pages, JSON output
odnelazm sittings --house senate --all -o json

# Members
odnelazm all-members 12th-parliament -o json

# Member profile, with full activity and bills
odnelazm profile <member-url> --all-activity --all-bills -o json
```

Output as a table, JSON, Parquet, or CSV (Parquet/CSV for future Hugging Face dataset exports).

The Fun Part: an MCP Server

- Wrap the scraper API as **tools** an LLM can call directly
- Crate: `rmcp`, the official Rust MCP SDK
- Same `HansardScraper`, now driven by an AI client instead of a CLI

With this, an LLM can fetch sittings, members, and member profiles straight from a prompt.

McpServer

```
#[derive(Debug, Clone)]
pub struct McpServer {
    scraper: HansardScraper,
    tool_router: ToolRouter<Self>,
}

#[tool_router]
impl McpServer {
    #[tool(
        name = "get_sitting",
        description = "Fetch a sitting: sections, subsections, \
            contributions, and procedural notes."
    )]
    pub async fn get_sitting(
        &self,
        Parameters(params): Parameters<GetSittingParams>,
    ) -> Result<String, McpError> { /* ... */ }

    // + list_sittings, list_members, get_all_members, get_member_profile
}
```


MCP: Current Limitations

- **Scrapes on demand:** every MCP call hits the live site directly
- **Context overload:** a full sitting as JSON can be huge
- Querying a wide date range can blow past a model's context window

Workarounds

- Interact with 1-2 (at most 3) sittings at a time
- Use a model with a large context window (1M tokens)
- Once it's in Postgres (next section), MCP tools could query the database directly instead of re-scraping

Try it yourself: point any MCP-compatible client (Claude, ChatGPT, etc.) at `https://odnelazm.c12i.xyz/sse` and start querying live Hansard data.

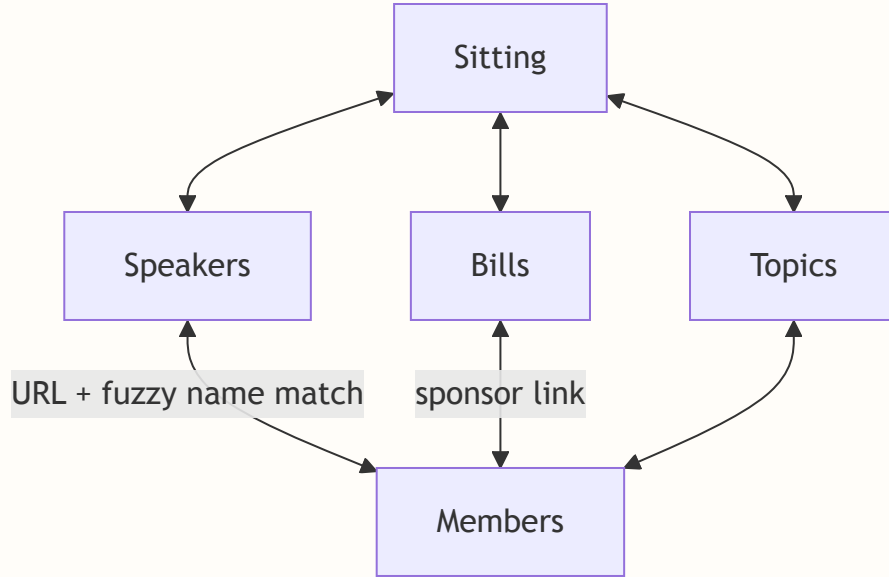
Act 3

odnelazm-ingest: Storing It All

From Scraped Data to a Database

- Store member profiles
- Store sittings
- Build associations:
 - Sitting ↔ Bills
 - Sitting ↔ Topics
 - Sitting ↔ Speakers
 - Speakers ↔ Member profiles
- Built around a storage **trait**: swap the backend without touching the pipeline

Data Associations



Every sitting becomes a node connected to bills, topics, and the people who spoke, queryable from any direction.

The DataStore Trait

```
pub trait DataStore: Send + Sync {
    async fn migrate(&self) → Result<()>;

    // Settings
    async fn upsert_sitting(&self, sitting: &HansardSitting) → Result<Uuid>;
    async fn list_ingested_urls(&self) → Result<Vec<String>>;

    // Speakers, bills, topics, and their associations
    async fn upsert_speaker(&self, speaker: &SpeakerRecord) → Result<Uuid>;
    async fn link_speaker_to_sitting(&self, speaker_id: Uuid, sitting_id: Uuid, count: u32) → Result<()>;
    async fn upsert_bill(&self, bill: &BillRecord) → Result<Uuid>;
    async fn upsert_topic(&self, topic: &TopicRecord) → Result<Uuid>;

    // Members
    async fn upsert_member(&self, member: &MemberRecord) → Result<Uuid>;
    async fn link_speakers_to_members(&self, parliament: &str) → Result<u64>;

    // ... + AI-generated summary storage for the enrich step
}
```

One implementation today: `PostgresStore`, using `sqlx`.

IngestPipeline

```
pub struct IngestPipeline<S: DataStore> {
    scraper: HansardScraper,
    store: S,
    embedder: Option<Arc<dyn Embedder>>,
    pub summarizer: Option<Arc<dyn Summarizer>>,
    pub metrics: Option<Arc<dyn MetricsSink>>,
}

impl<S: DataStore> IngestPipeline<S> {
    /// Ingest a single fully-fetched sitting. All other ingest methods
    /// funnel through here.
    async fn ingest_sitting(&self, sitting: HansardSitting) → Result<IngestStats> {
        let sitting_id = self.store.upsert_sitting(&sitting).await?;

        for (speaker, count) in extract_speakers(&sitting) {
            let id = self.store.upsert_speaker(&speaker).await?;
            self.store.link_speaker_to_sitting(id, sitting_id, count).await?;
        }
        // ... bills and topics follow the same extract → upsert → link shape

        Ok(stats)
    }
}
```

CLI: Running the Pipeline

```
# Ingest everything
odnelazm-pipeline ingest

# Ingest a date range, skip member import
odnelazm-pipeline ingest --start-date 2026-01-01 --end-date 2026-03-31 --skip-members

# Also fetch full member profiles (bio, photo, committees, voting record)
odnelazm-pipeline ingest --import-profiles

# Custom database + Prometheus metrics
odnelazm-pipeline --database-url postgres://user:pass@host/db --metrics-url http://localhost:9091 ingest
```

Designed to run unattended, metrics pushed to Prometheus via `--metrics-url` .

Act 4

AI Enrichment, Sustainably

Bringing LLMs Back, Carefully

Bunge Bits ran on cloud LLMs for transcription and summarization.

- Cloud LLMs are expensive at scale: hours of audio, every day, billed per token
- No funding model meant the costs were unsustainable long term
- AI-generated transcripts as the *source of truth* proved fragile, official Hansards are a far better foundation
- But summarization itself, layered on top of real transcripts, is still genuinely useful

A Sustainable Enrichment Strategy

- Run models **locally** via LM Studio, on consumer hardware
- No cloud API bills, no per-token meter running
- Trade-off accepted: throughput is slower
- Every summary records **which model** generated it, so re-enrichment later with a better model is just another batch job

Enrichment Targets

```
#[derive(ValueEnum, Clone)]
enum EnrichTarget {
    BillMentions, // a bill's appearance within one sitting
    BillJourneys, // a bill's full journey across all sittings
    BillSpeakers, // a member's remarks on a bill, in one sitting
    Topics,       // a topic's appearance within one sitting
    TopicSpeakers, // a member's remarks on a topic, in one sitting
    Sittings,     // a rich summary of the whole sitting
}
```

Six targets, one shape: find pending rows, prompt the model, store the summary along with the model name.

One Target Up Close: Bill Mentions

```
/// A bill_mention row needing a node-level summary. Carries the full
/// sitting transcript as JSON for context.
pub struct PendingBillAppearanceSummary {
  pub bill_mention_id: Uuid,
  pub bill_name: String,
  pub bill_number: Option<String>,
  pub stage: Option<String>,
  pub section_title: String,
  pub date: NaiveDate,
  pub house: String,
  pub session_type: String,
  pub sitting_raw_json: serde_json::Value,
}
```

One row per bill, per sitting it appears in: "here's everything that happened in this sitting, write up how this bill featured in it."

See it live: [a bill's journey on Bunge Hub](#), built from these per-sitting summaries.

Picking the Right Model per Target

- `bill-mentions` → `qwen3.5-9b`, a reasoning model
 - Context is the full sitting: a bigger context window, strong instruction-following, and more substantial summaries pay off here
- `bill-speakers` and `topic-speakers` → `google/gemma-4-e2b`
 - Context is scoped to just that bill's or topic's mentions, so a smaller, lighter-reasoning model handles it well
- Right-size the model to the target: reasoning where context is broad, fast and light where context is narrow

Summarizer: Another Trait Boundary

```
/// Generate a brief summary from a prompt. Implement this trait with
/// your preferred LLM provider.
#[async_trait]
pub trait Summarizer: Send + Sync {
    async fn summarize(&self, prompt: &str) -> Result<String>;
}
```

```
pub struct LmStudioSummarizer {
    client: Client,
    base_url: String, // e.g. http://127.0.0.1:1234
    model: String,    // e.g. "google/gemma-4-e4b"
    temperature: Option<f32>,
    metrics: Option<Arc<dyn MetricsSink>>,
}

#[async_trait]
impl Summarizer for LmStudioSummarizer {
    async fn summarize(&self, prompt: &str) -> Result<String> {
        self.complete(prompt).await // POST to LM Studio's /api/v1/chat
    }
}
```

Same trait-based pattern as `DataStore`: today it's LM Studio, tomorrow it could be any local or hosted model.

The Enrichment Loop

```
async fn run_bill_mentions(&self, store: &PostgresStore, summarizer: &dyn Summarizer) {
    loop {
        let pending = store.pending_bill_appearance_summaries(self.batch).await?;
        if pending.is_empty() {
            break;
        }

        for chunk in pending.chunks(self.concurrency) {
            let tasks = chunk.iter().map(|p| async move {
                let prompt = prompts::bill_appearance_prompt(p);
                (p.bill_mention_id, summarizer.summarize(&prompt).await)
            });

            for (id, result) in future::join_all(tasks).await {
                if let Ok(summary) = result {
                    store.store_bill_appearance_summary(id, &summary, &self.model).await.ok();
                }
            }
        }
    }
}
```

Fetch a batch, fan out `concurrency` requests to the local model, persist, repeat until nothing is pending.

CLI: Running Enrichment

```
# Summarize a bill's appearance in a sitting (full transcript context)
odnelazm-pipeline enrich bill-mentions --model qwen3.5-9b

# Summarize a member's remarks on a bill, one sitting at a time
odnelazm-pipeline enrich bill-speakers --model google/gemma-4-e2b

# Point at a local LM Studio server, tune batching and concurrency
odnelazm-pipeline enrich sittings \
  --llm-url http://127.0.0.1:1234 \
  --model qwen/qwen3-30b \
  --batch 5 \
  --concurrency 2

# Same --database-url / --metrics-url flags as ingest
odnelazm-pipeline --database-url postgres://user:pass@host/db --metrics-url http://localhost:9091 enrich <enrichment-ta
```

Same `odnelazm-pipeline` binary as ingestion, run as a separate, unhurried batch job.

Act 5

Bunge Hub: Putting It in Front of People

Bunge Hub

The public face of all this data.

- Browse sittings, bills, topics, and members
- Search across sittings, bills, and topics
- Same data pipeline from Acts 2 and 3, now in front of citizens

bunge-hub.mwananchi.tech



Closing the Loop: Bunge Bits, Locally

- Bunge Bits is still useful: it can surface summaries within hours, official Hansards take 1-2 weeks to publish
- Act 1 ended on the cost of cloud Whisper and cloud LLMs as the limiting factor
- Act 4 showed local models are good enough for enrichment, on consumer hardware
- Next: bring that back to Bunge Bits itself, local Whisper for transcription, local LLMs for summaries

What's Next

- **Bulk writes** for ingest and enrich: fewer round trips, faster backfills
- **Scheduled ingestion** via apalis-cron: keep Bunge Hub fresh automatically
- Export to Hugging Face datasets (Parquet/CSV)
- **Enrichment is ongoing**: bill summaries and topic-mentions still have a backlog of pending rows
- **Bunge Bits revival**: local Whisper and local LLMs, as covered a moment ago

Anyone can run `odnelazm-pipeline enrich`. Want to help chip away at the backlog? Come talk to me and I'll share database access.

Why Rust for Civic Tech?

- **Traits** decouple scraping, storage, and transcription, so any layer can be swapped or mocked
- **Async** lets us scrape, transcribe, and ingest concurrently
- **Type safety** tames messy, inconsistent government HTML
- **Single static binaries**: easy to deploy, easy to run as a cron job

Thank You

github.com/mwananchi-tech

Bunge Bits · odnelazm · Bunge Hub

Questions?